# Machine Learning – CDS201

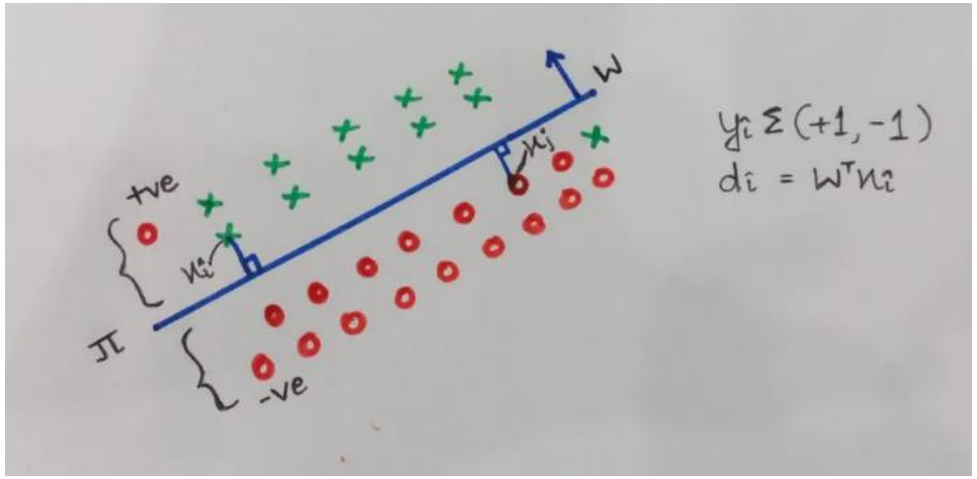
Module 3: Classification Models

# Logistic Regression

- It is a type of classification algorithm that predicts a discrete or categorical outcome.
- For example, we can use a classification model to determine whether a loan is approved or not based on predictors such as savings amount, income and credit score.
- Logistic regression, like linear regression, is a type of linear model that examines the relationship between predictor variables (independent variables) and an output variable (the response, target or dependent variable). The key difference is that linear regression is used when the output is a **continuous value**—for example, predicting someone's credit score. Logistic regression is used when the outcome **is categorical**, such as whether a loan is approved or not.

# Let’s start to frame the model for logistic regression from linear regression

- Assume : 2 – class problem : Binary classification
  - Task: to find a **hyperplane (n-Dimensional) or line (2-D)**that best separates the classes.
  - Imagine, we are given a set of two classes as shown below, positive points shown by ‘x’ and negative points by ‘o’ respectively. So, the task is to find the best line or plane that best separates positive points from negative.



# Logistic regression modelling

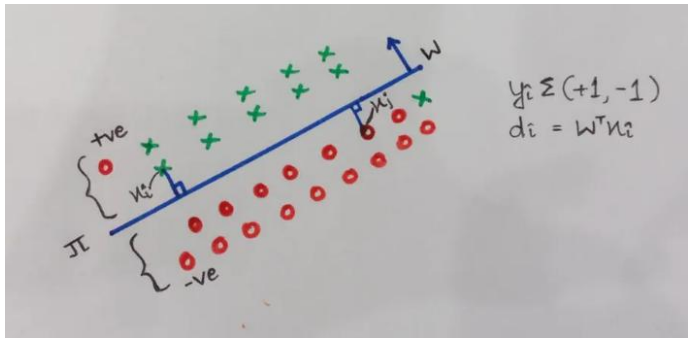
- **Assumption**: the classes are perfectly or almost perfectly linearly separable.

- y (class label)= +1 :positive points , -1: negative points

- Distance (di) of any point xi from the plane( $\pi$): di=wTxi + b/||w||
  - wT -> vector of (w1,w2) , b = intercept
  - Assume: the plane passes through origin (b=0) & **w is a unit vector ||w|| =1 to keep it simple**. Then,

  - di=wTxi

# Logistic regression modelling




- When w and xi are in the same direction

  i.e. wTxi>0, then y= +1

- when w and xj pointing in the opposite direction

  i.e. wTxj<0, then y = -1

**Note to remember:** *Basically, every point which is in the same direction as w is a positive point (dot product is positive) and every point in the opposite direction (dot product is negative) is a negative point. This is how the classifier works.*

**Recollecting the task:** find a plane that best separates the two classes.

# Logistic regression modelling

- let us have a look at some cases first:

- **Case 1:**
  - We are given the true class label as yi=+1 i.e. a positive point
  - and wTxi>0 i.e. classifier indicating it's a positive point
  - Then **yiwTxi >, 0** which means the plane is correctly classifying the point.
- **Case 2:**
  - We are given the true class label as yi=-1 i.e. a negative point
  - and wTxi<0 i.e. classifier indicating its a negative point
  - Then **yiwTxi > 0** which means the plane is correctly classifying the point.

# Logistic regression modelling

- **Case 3:**
  - We are given the true class label as yi=+1 i.e. a positive point
  - and wTxi<0 i.e. classifier indicating its a negative point
  - Then **yiwTxi < 0** which means the plane is incorrectly classifying the point.
- **Case 4:**
  - We are given the true class label as yi=-1 i.e. a negative point
  - and wTxi>0 i.e. classifier indicating its a positive point
  - Then **yiwTxi < 0** which means the plane is incorrectly classifying the point.

# Logistic regression modelling

- when **yiwTxi > 0,** we are correctly classifying the points, and when **yiwTxi < 0** we are incorrectly classifying the points.

- For a classifier to perform well, we need to maximize correctly classified points and minimize incorrectly classified points. In short, we need to have as many points as possible to have **yiwTxi > 0.**

# Logistic regression modelling

- This is what we need to achieve and in order to do that, we need to find the optimal w, which will solve this maximization problem, as both y and x are fixed. So this is a mathematical problem which we have achieved that we would term as **'Optimization Problem'.**

$$w^* = argmax\ \left(\sum_{i=1}^{n} y_i\, w^T x_i\right)$$

- There are several hyperplanes, and for each plane, there is a unique 'w'. So we need to find the optimal w i.e. the maximum value and that w would give us our best plane which would be our decision surface.

# The problem associated with this model

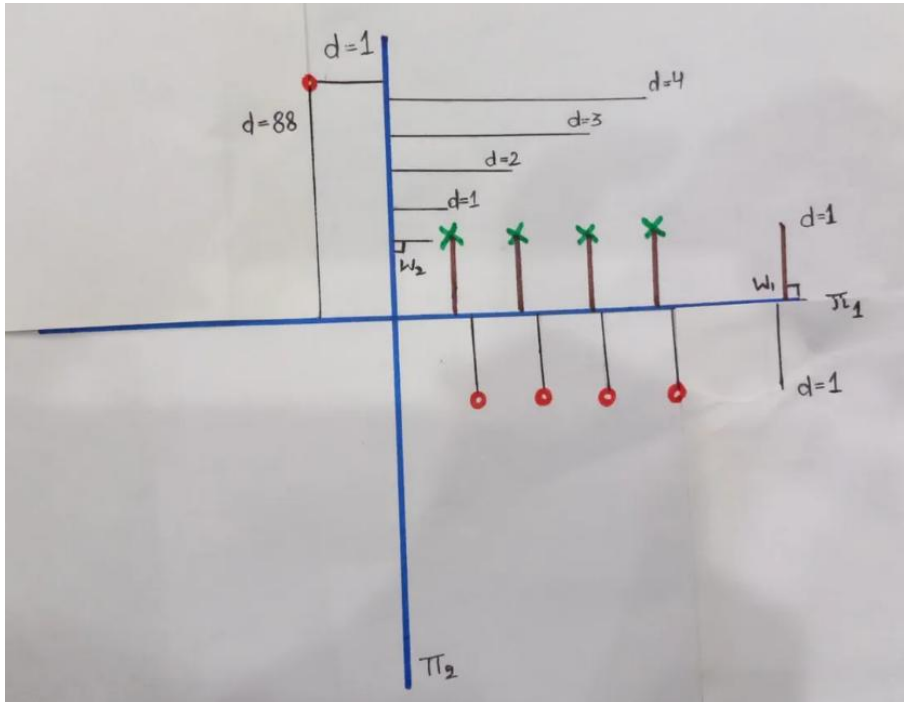
- Recollect: **wTxi** is the distance from xi to plane and yi is either+1 or -1.

- The maximizing sum of signed distances is not outlier prone and it can get impacted by outliers. In some cases, even a single outlier can have a large impact and can result in the model performing badly.

- In the above example, we see that for $\pi_1$, we have 8 correctly classified points and one incorrectly but the sum of signed distances would be too -80. (+1(*4) +1(*4)+ 88(-1) ).

- Talking about $\pi_2$, we can see that it only classifies 5 points correctly but gives a sum of signed distance as 1 (1+1+2+3+4 -1–2–3–4).

- Though $\pi_1$ gives more correctly classified points due to **one extreme outlier**, our optimization problem says $\pi_2$ is better.



We have an outlier

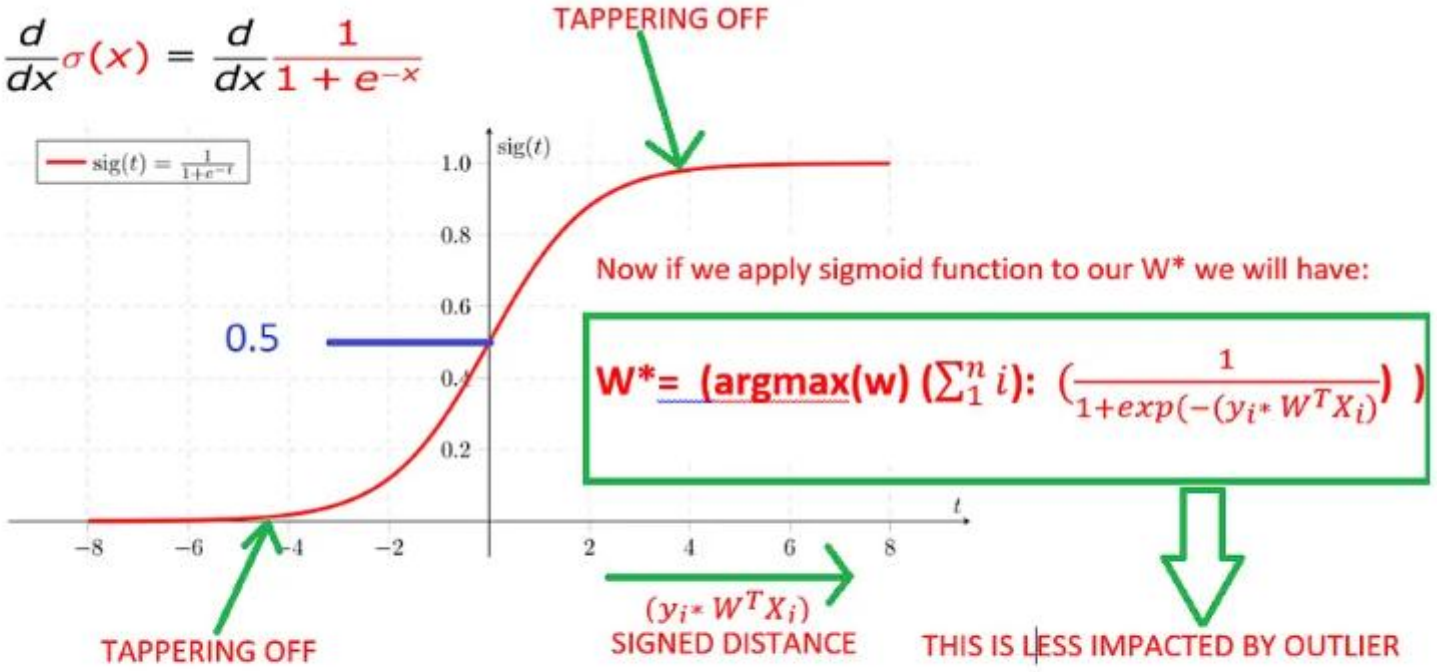
made with mematic
We have squashing

- In order to deal with this problem, we need to modify the optimizing equation by applying a technique called **squashing**.

- The idea behind squashing is that if the distance of a point from the plane is small then we will use it as it is, but if the distance is large, then convert it into a small value.

- We will do that by applying a sigmoid function to the equation which will help us to deal with the outlier problem. It would convert our signed distances which are in any range (-infinity, +infinity) to [0,1]

$$\sigma'(x) = \frac{d}{dx}\sigma(x) = \frac{d}{dx}\frac{1}{1+e^{-x}}$$




Now if we apply sigmoid function to our W* we will have:

$$W^* = (\text{argmax}(w)\ (\textstyle\sum_1^n i):\ (\frac{1}{1+exp(-(y_{i*} W^T X_i)}))$$

$$W^* = \text{argmax}(\textstyle\sum_{i=1}^{n} 1/(1+\exp(-Y_i W^T X_i)))$$

- **Why sigmoid function? The reasons are:**
- It is easily differentiable.
- It offers a probabilistic interpretation.
- It gives linear behavior when xi’s are small whereas it provides tapering behavior when xi’s are large.

- Now this is a Classifier with a threshold as 0.5

  - If sigmoid(wT.x)>0.5, then class label = 1 in this case.
    If sigmoid(wT.x)<0.5, then class label = 0 in this case.

  - Whether the point is correctly classified or not is dependent on the sign of y*(wT.x)
    If y*(wT.x) = +ve, then the point is correctly classified.
    If y*(wT.x) = -ve, then the point is misclassified.

| Aspect | Linear Regression | Logistic Regression |
|---|---|---|
| **Problem Type** | Regression (predicting continuous values) | Classification (predicting categorical outcomes) |
| **Output** | Continuous numerical values (can be any real number) | Probabilities (0 to 1) converted to discrete class labels |
| **Function Used** | Linear function: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots$ | Sigmoid/Logistic function: $\sigma(z) = 1/(1+e^{-z})$ |
| **Output Range** | $-\infty$ to $+\infty$ (any real number) | 0 to 1 (probability) |
| **Use Case Example** | Predicting house prices, stock prices, and temperature | Email spam detection, disease diagnosis, and customer churn |
| **Evaluation Metrics** | R-squared, MSE, RMSE, MAE | Accuracy, Precision, Recall, F1–score, AUC-ROC |

# Test your knowledge

Why is a linear regression model's output a poor predictor of probability?

A. It only has one weight per feature.

B. It only has one output value.

C. The bias parameter skews the output value.

D. Its predictions are not restricted to values between 0 and 1.

# Test your knowledge

- True or false: A sigmoid function never outputs the value 0 or the value 1.

A. True

B. False

# Test your knowledge

- True or false: Applying regularization is less important when training logistic regression models than it is for training linear regression models.

A. True

B. False

# Types of Logistic Regression Models

- Binary Logistic Regression
  - **Examples:** Email (Spam/Not Spam), Medical diagnosis (Disease Present/Absent), Customer behavior (Churn/Retain), Loan approval (Approved/Rejected).
- Multinomial Logistic Regression
  - **Examples:** Transportation mode (Bus, Train, Car, Bike), Product preference (Brand A, B, C), Image classification (Cat, Dog, Bird).
- Ordinal Logistic Regression
  - **Examples:** Customer satisfaction (Very Unsatisfied to Very Satisfied), Disease severity (Mild, Moderate, Severe), Education level (High School, Bachelor’s, Master’s, PhD), Credit rating (Poor, Fair, Good, Excellent).

- **When to Choose:**
  - **Binary:** Two categories only
  - **Multinomial:** Three+ unordered categories
  - **Ordinal:** Three+ ordered categories

# Multinomial Logistic Regression

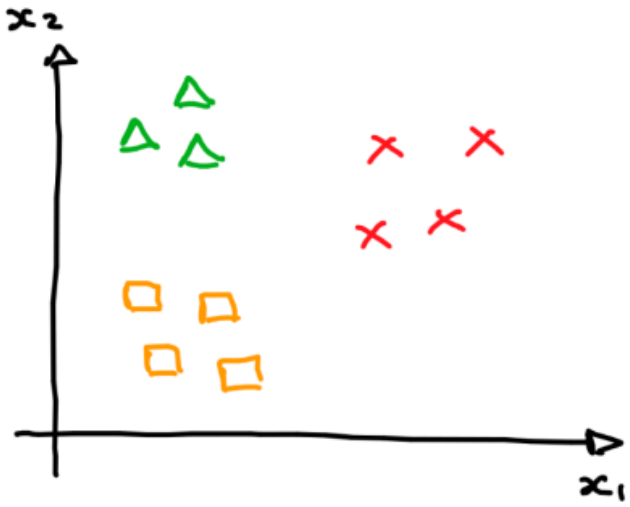
- Multiclass logistic regression is a machine learning method used when the target variable has more than two categories. Unlike binary logistic regression which predicts two outcomes it helps classify data into three or more classes.

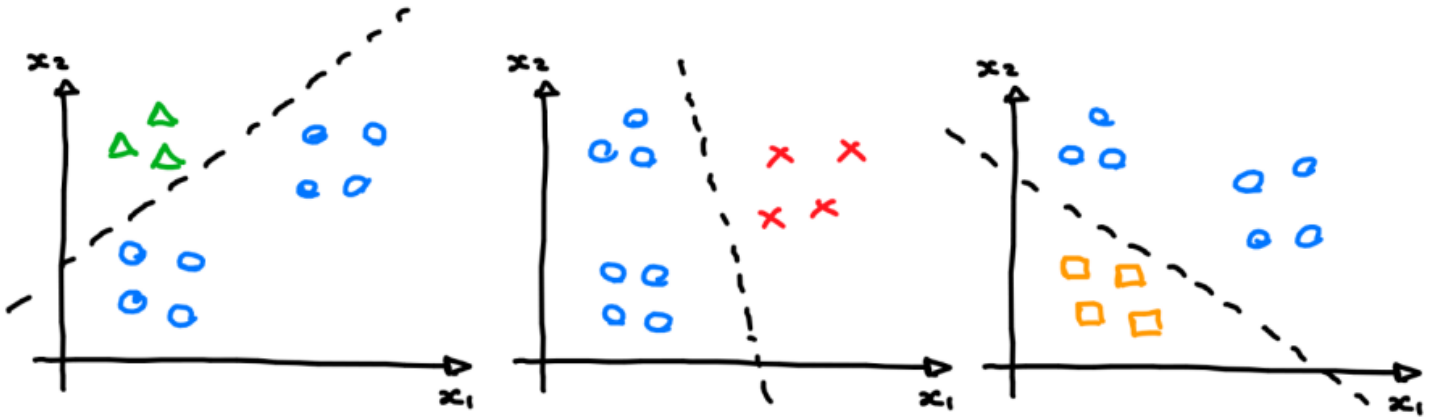
- One Vs Rest
- One Vs One
- SoftMax

# One Vs Rest

- One-vs-rest (OvR for short, also referred to as One-vs-All or OvA) is a heuristic method for using binary classification algorithms for multi-class classification.
- It involves splitting the multi-class dataset into multiple binary classification problems. A binary classifier is then trained on each binary classification problem and predictions are made using the model that is the most confident.

# One-vs-rest classification




Strategy: Train three classifiers with $y \in \{0, 1\}$, where each classifier considers another class as the positive class.

We then get three classification models:

$f_1(\underline{x}; \underline{w}_1)$ 1: △

$f_2(\underline{x}; \underline{w}_2)$ 2: ×

$f_3(\underline{x}; \underline{w}_3)$ 3: □

Final predictions: $\arg\max_k f_k(\underline{x}; \underline{w}_k)$

- For example, given a multi-class classification problem with examples for each class '*red*,' '*blue*,' and '*green*'. This could be divided into three binary classification datasets as follows:
  - **Binary Classification Problem 1**: red vs [blue, green]
  - **Binary Classification Problem 2**: blue vs [red, green]
  - **Binary Classification Problem 3**: green vs [red, blue]

  - A possible downside of this approach is that it requires one model to be created for each class. For example, three classes requires three models. This could be an issue for large datasets (e.g. millions of rows), slow models (e.g. neural networks), or very large numbers of classes (e.g. hundreds of classes).

# One-Vs-One for Multi-Class Classification

- One-vs-One (OvO for short) is another heuristic method for using binary classification algorithms for multi-class classification.
- Like one-vs-rest, one-vs-one splits a multi-class classification dataset into binary classification problems. Unlike one-vs-rest that splits it into one binary dataset for each class, the one-vs-one approach splits the dataset into one dataset for each class versus every other class.

- For example, consider a multi-class classification problem with four classes: ‘*red*,’ ‘*blue*,’ and ‘*green*,’ ‘*yellow*.’ This could be divided into six binary classification datasets as follows:
- **Binary Classification Problem 1**: red vs. blue
- **Binary Classification Problem 2**: red vs. green
- **Binary Classification Problem 3**: red vs. yellow
- **Binary Classification Problem 4**: blue vs. green
- **Binary Classification Problem 5**: blue vs. yellow
- **Binary Classification Problem 6**: green vs. yellow

Each binary classification model may predict one class label and the model with the most predictions or votes is predicted by the one-vs-one strategy.

- This is significantly more datasets, and in turn, models than the one-vs-rest strategy described in the previous section.

- The formula for calculating the number of binary datasets, and in turn, models, is as follows:

$$(NumClasses * (NumClasses – 1)) / 2$$

- We can see that for four classes, this gives us the expected value of six binary classification problems:

$$(NumClasses * (NumClasses – 1)) / 2 = 6$$

# Softmax regression

- Softmax regression (or multinomial logistic regression) is a generalization of logistic regression to the case where we want to handle multiple classes using a single model.

- Softmax regression allows us to handle $y(i) \in \{1,\ldots,K\}$, where K is the number of classes.

- In the softmax regression setting, we are interested in multi-class classification (as opposed to only binary classification), and so the label y can take on K different values, rather than only two. Thus, in our training set $\{(x(1),y(1)),\ldots,(x(m),y(m))\}$ , we now have that $y(i) \in \{1,2,\ldots,K\}$.

- Given a test input x, we want our hypothesis to estimate the probability that $P(y=k|x)$ for each value of $k=1,\ldots,K$. I.e., we want to estimate the probability of the class label taking on each of the K different possible values. Thus, our hypothesis will output a K-dimensional vector (whose elements sum to 1) giving us our K estimated probabilities.

# Softmax regression model

- For binary logistic regression we had $f(\mathbf{x};\mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \dfrac{1}{1+e^{-\mathbf{w}^\top \mathbf{x}}}$ with $y \in \{0,1\}$.
- We interpreted the output as $P(y=1|\mathbf{x};\mathbf{w})$, implying $P(y=0|\mathbf{x};\mathbf{w}) = 1 - f(\mathbf{x};\mathbf{w})$.
- For the multiclass setting we now have $y \in \{1,2,\ldots,K\}$.
- **Idea:** Instead of just outputting a single value for the positive class, let's output a vector of probabilities for each class:

$$\boldsymbol{f}(\mathbf{x};\mathbf{W}) = \begin{bmatrix} P(y=1|\mathbf{x};\mathbf{W}) \\ P(y=2|\mathbf{x};\mathbf{W}) \\ \vdots \\ P(y=K|\mathbf{x};\mathbf{W}) \end{bmatrix}$$

- We will now build up to a model that does this.

# Softmax regression model

- Each element in $\boldsymbol{f}(\mathbf{x};\mathbf{W})$ should be a "score" for how well input $\mathbf{x}$ matches that class.
- For input $\mathbf{x}$, let's set the score for class $k$ to $\mathbf{w}_k^\top \mathbf{x}$.
- But probabilities need to be positive. So let's take the exponential: $e^{\mathbf{w}_k^\top \mathbf{x}}$.
- But probabilities need to sum to one. So let's normalise:

$$P(y=k|\mathbf{x};\mathbf{W}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^\top \mathbf{x}}}$$

- This gives us the softmax regression model:

$$\underline{f}(\underline{x};\underline{W}) = \frac{1}{\sum_{j=1}^{K} e^{\underline{w}_j^T \underline{x}}} \begin{bmatrix} e^{\underline{w}_1^T \underline{x}} \\ e^{\underline{w}_2^T \underline{x}} \\ \vdots \\ e^{\underline{w}_K^T \underline{x}} \end{bmatrix} = \text{softmax}(\underline{W}\,\underline{x})$$

**Parameters:**

Vectors $\underline{w}_1, \underline{w}_2, \ldots, \underline{w}_K$

Parameter matrix:

$$\underline{W} = \begin{bmatrix} - & (\underline{w}_1)^T & - \\ - & (\underline{w}_2)^T & - \\ & \vdots & \\ - & (\underline{w}_K)^T & - \end{bmatrix}$$

# Optimization

- Fit model using maximum likelihood. Equivalent to minimising the negative log likelihood:

$$
\begin{aligned}
J(\mathbf{W}) &= -\log L(\mathbf{W}) \\
&= -\sum_{n=1}^{N} \log P(y^{(n)} | \mathbf{x}^{(n)}; \mathbf{W}) \\
&= -\sum_{n=1}^{N} \sum_{k=1}^{K} \mathbb{I}\{y^{(n)} = k\} \log \frac{e^{\mathbf{w}_k^\top \mathbf{x}^{(n)}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^\top \mathbf{x}^{(n)}}}
\end{aligned}
$$

- Derivatives:

$$
\frac{\partial J(\mathbf{W})}{\partial \mathbf{w}_k} = -\sum_{n=1}^{N} \left( \mathbb{I}\{y^{(n)} = k\} - f_k(\mathbf{x}^{(n)}; \mathbf{W}\ ) \right) \mathbf{x}^{(n)}
$$

- Using these derivatives, we can minimise the loss using gradient descent.

# Relationship between binary and logistic Regression

- **Softmax for 2 classes** simplifies exactly to **sigmoid**.
  - Say we have 2 classes, with scores $z_1, z_2$:

$$P(y=1) = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$$

$$P(y=1) = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{e^{z_1}}{e^{z_2}\left(\frac{e^{z_1}}{e^{z_2}} + 1\right)}$$

$$P(y=1) = \frac{e^{z_1}}{e^{z_2}\left(\frac{e^{z_1}}{e^{z_2}} + 1\right)} = \frac{e^{z_1}/e^{z_2}}{\frac{e^{z_1}}{e^{z_2}} + 1}$$

$$\frac{e^{z_1}}{e^{z_2}} = e^{z_1 - z_2} = e^{z}$$

**Substitute** $z = z_1 - z_2$

$$P(y=1) = \frac{e^{z}}{e^{z} + 1} = \frac{1}{1 + e^{-z}}$$

- 2 classes: softmax = sigmoid
- >2 classes: softmax ≠ sigmoids (unless you change the task to multi-label)
- Sigmoid is independent; softmax is normalized and mutually exclusive.